SQL: Filtering Data for Security Investigations

Investigating login activity and targeting machines for security updates

Type	Hands-on lab (SQL)
Tools	SQL (MariaDB)
Skills	`SELECT` / `WHERE`, `AND` / `OR` / `NOT`, pattern matching with `LIKE` and `%`, log investigation
Source	Google Cybersecurity Professional Certificate, Course 4 (*Tools of the Trade: Linux and SQL*)

Project description

As a security analyst, I queried two tables in the organisation's database. One was `log_in_attempts`, which records the date, time, country and success of each login. The other was `employees`, which records each employee's department, office and machine. I used SQL filters to investigate suspicious login activity and to find which employee machines needed security updates.

1. Failed login attempts after business hours

```sql
SELECT *
FROM log_in_attempts
WHERE login_time > '18:00'
  AND success = FALSE;
```

A possible incident happened after business hours. This query returns only attempts that match **both** conditions: the login time is after 18:00, **and** the attempt failed (`success` is `0`/`FALSE`). A cluster of failed logins after hours can point to password guessing or a compromised account.

2. Login attempts on specific dates

```sql
SELECT *
FROM log_in_attempts
WHERE login_date = '2022-05-09'
   OR login_date = '2022-05-08';
```

A suspicious event happened on 2022-05-09. Using `OR` returns attempts from **either** that day **or** the day before, so activity leading up to the event is included too.

3. Login attempts outside Mexico

```sql
SELECT *
FROM log_in_attempts
WHERE NOT country LIKE 'MEX%';
```

The `country` column stores Mexico as both `MEX` and `MEXICO`. The pattern `'MEX%'` matches both, because `%` stands for any sequence of characters. `NOT` then excludes them, which leaves every attempt from outside Mexico.

4. Marketing employees in the East building

```sql
SELECT *
FROM employees
WHERE department = 'Marketing'
  AND office LIKE 'East%';
```

Office values look like `East-170` or `North-434`. `LIKE 'East%'` matches any office in the East building, and `AND` limits the results to the Marketing department. This gives the exact list of machines that need the update.

5. Employees in Sales or Finance

```sql
SELECT *
FROM employees
WHERE department = 'Sales'
   OR department = 'Finance';
```

A different update was needed for two departments. `OR` returns employees who are in **either** one. (`WHERE department IN ('Sales', 'Finance')` gives the same result.)

6. All employees not in IT

```sql
SELECT *
FROM employees
WHERE NOT department = 'Information Technology';
```

IT machines had already been updated. `NOT` returns everyone else, so the update reaches every remaining machine. (`department <> 'Information Technology'` is equivalent.)

Summary

I wrote six filtered queries using `AND`, `OR`, `NOT` and `LIKE` with wildcards. They isolated failed after-hours logins, focused on suspicious dates, excluded an expected country, and produced exact lists of machines for targeted patching. Being able to quickly pull a precise subset from a large log table is a core skill in SOC investigations and vulnerability management.

All queries are also available in `queries.sql`.